栈、队列和数组 3

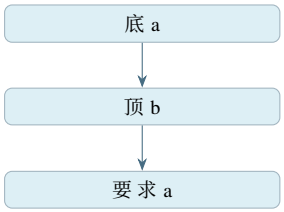
章节脉络



栈与队列都保存线性序列，差别在于允许从哪一端加入和取出。判断一个操作序列是否合法，必须保留操作过程中的状态；只检查输入、输出是否含有相同元素，不能排除违反栈顶或队头约束的情况。

最小反例·数量相同仍不合法

顺序输入 a、b、c，要先输出 c 就必须把三者全部入栈；弹出 c 后栈顶为 b，接着要求 a 便失败。输出 c、a、b 与输入含相同元素，却违反栈顶约束。



c 已弹出，b 挡在 a 上方

3.1 栈的后进先出与合法序列

栈只允许在同一端插入和删除，这一端称为栈顶，另一端称为栈底。入栈称 `push`，出栈称 `pop`。栈为空时不能出栈；固定容量栈满时不能继续入栈。若中途不出栈，一批元素的出栈次序是入栈次序的逆序；若允许交替入栈和出栈，输出就可能有多种。

检验给定输出的方法是按目标逐个模拟：若当前栈顶不是所需元素，继续把尚未入栈的元素依次压入；一旦所需元素到顶，就弹出。若输入已用尽而栈顶仍不符，输出不合法。这个过程也给出实现该输出所需的最小容量，即模拟中出现的最大栈深。

3.1.1 例 3-1　队列不会改变栈的输出次序

例 3.1.1 (配套例题)

配套例题：栈 S 和队列 Q 初始均为空，元素 `a,b,c,d,e,f,g` 依次入栈 S。每个元素出栈后立即进入队列 Q，7 个元素出队顺序为 `b,d,c,f,e,a,g`，则栈容量至少是（　）。

A. 1　　B. 2　　C. 3　　D. 4

解析。队列保持进入它的先后次序，因此它的输出就是栈的输出。按目标模拟，栈内状态按栈底到栈顶写：

需要输出	必须先执行的入栈	弹出后栈内剩余
b	a、b	a
d	c、d	a、c
c	无	a
f	e、f	a、e
e	无	a
a	无	空
g	g	空

错项排除·共享栈只在相邻时满

两栈从数组两端向中间增长，未用单元恰位于两栈顶之间。两栈顶相邻已经没有空位；等到相等才判满，就已让两个栈占用同一个位置。

压入 d 或 f 后，栈深达到 3；表中过程也说明 3 个单元足够，答案为 **C**。不能把队列当作另一处可以随意调整次序的缓冲区。

若只用入栈 `I`、出栈 `O` 表示操作，一段从空栈出发并回到空栈的合法序列须满足：两类操作的总数相等，且每个前缀中 `O` 的个数不超过 `I`。对 n 个不同元素，合法出栈序列数为卡特兰数 $C_n = \frac{1}{n+1}\binom{2n}{n}$；这一公式用于计数，不代替具体容量与给定序列的模拟。

输入为 `a,b,c,d` 时，若输出以 `c,d` 开头，弹出 c 后栈中为 a、b，随后必须压入并弹出 d，剩余只能 b、a，因此仅有 `c,d,b,a` 一种。若题目指定的是栈的输出而让输入未知，也要反推中间栈顶。例如第 3 个输入是 1，输出从 1 递增，1 弹出时第 2 个输入压在第 1 个输入上方；若第 1 个是 2，就要先弹出上面的另一个值才能到 2，与“接下来输出 2”矛盾，所以第 1 个不可能是 2。

3.2 队列与双端队列

队列只在队尾加入、在队头删除，遵循先进先出。它适合保存等待依次处理的对象，如按发现顺序等待扩展的图顶点。队头与队尾是逻辑角色，不能凭数组下标的大小判断先后：循环队列绕回后，队尾下标可以小于队头下标。

双端队列允许在两端插入和删除。输入受限双端队列只允许一端输入，两端均可输出；输出受限双端队列允许两端输入，只允许一端输出。判断输出时应明确属于哪一种，不能直接套普通队列或普通栈的限制。双端队列能够模拟栈与队列，但这不表示它能按任意次序输出一组顺序输入的元素。

以 1、2、3、4 依次输入为例，普通栈有 14 种合法输出。输入受限双端队列有 22 种，不能输出 `4,2,1,3` 和 `4,2,3,1`：先输出 4 意味着 1、2、3 已按次序留在队列中，2 在中间，下一次无法取出。输出受限双端队列也有 22 种，不能输出 `4,1,3,2` 和 `4,2,3,1`：4 第一次输出前，前三个元素只能通过两端依次接入，其剩余次序只能为 123、213、312 或 321，不能为 132 或 231。两种受限队列的非法序列集合并不相同。

3.3 顺序栈、共享栈与循环队列

顺序栈存于 `A[0..M-1]`。本章令 `top` 指向当前栈顶，初值 -1：空条件 `top==-1`，满条件 `top==M-1`；入栈为 `A[++top]=x`，出栈为 `x=A[top--]`。若题目改用“`top` 指向下一个空位”，空条件就为 0，入栈与出栈的增减位置也随之改变。先画一个元素的状态，再写公式，通常比背四个表达式可靠。

两个栈共享一个数组时，可让一端从左向右长、另一端从右向左长。初值 `top1=-1, top2=M`，栈满当且仅当 `top2-top1==1`。未用空间是两栈顶之间的单元，某个栈已用多少并不单独决定全局是否栈满。

入栈前必须先检查两栈顶是否相邻，不能等两个栈顶已经重合才报告满。

辨析·循环队列先定约定

牺牲一个单元时，空为 front=rear，满为 (rear+1)%MaxSize=front；其他设计的条件需另算。

3.3.1 循环队列先立约定再计算

采用长度为 M 的数组，令 `front` 指向队头元素，`rear` 指向下一次入队位置。出队后 `front=(front+1)%M`，入队后 `rear=(rear+1)%M`。若只有这两个下标，空与完全占满都会出现 `front==rear`，无法区分。

牺牲一个单元时，约定空条件为 `front==rear`，满条件为 `(rear+1)%M==front`，可保存的最大元素数是 $M-1$。当前长度为

$$\text{len} = (rear - front + M) \bmod M.$$

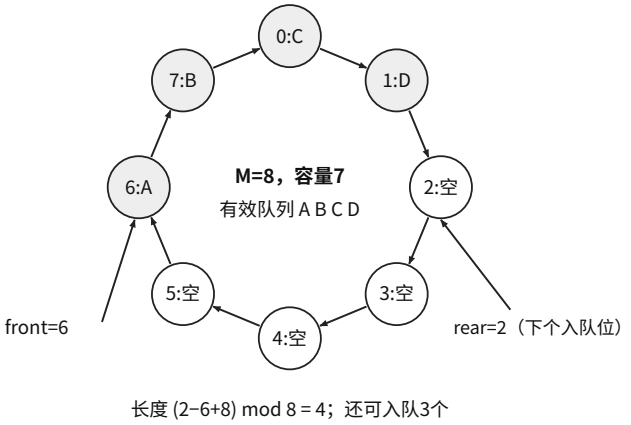


Figure 3.1: 数组环绕时，下标大小不代表队列先后；本图采用牺牲一个单元

若要求 M 个单元全部可用，可以增加 `size` 计数，或增加标志，区分最近一次使二者相等的是入队还是出队。此时 “空” 和 “满” 的判断依赖新增成员，不能仍照搬牺牲单元的满条件。

3.3.2 例 3-2　换了 front 含义，长度公式是否改变

例 3.3.1 (配套例题)

〖3.2-006〗循环队列存于数组 `A[21]`，`front` 指向队首元素的前一个位置，`rear` 指向队尾元素。当前 `front=8, rear=3`，队列长度为（　）。

A. 5　B. 6　C. 16　D. 17

解析。有效元素从下标 9 开始，经过 9 至 20，再经过 0 至 3，共 $12+4=16$ 个，答案为 **C**。公式仍可写成 $(3-8+21) \bmod 21 = 16$，是因为与前一约定相比，两个指针的含义都错开了一个位置。若只改变一个指针的含义，公式就未必不变。数组是 `A[0..n]` 时实际有 $n+1$ 个单元，取模也必须使用 $n+1$。

另一种常见表示令 front 仍指队头，而 rear 指当前队尾。可令初始 front=0、rear=M−1；入队先推进 rear 再写入，出队先读取 front 再推进。牺牲一个单元时，空条件是 `(rear+1)%M==front`，满条件是 `(rear+2)%M==front`，长度为 `(rear-front+1+M)%M`。例如 M=5、front=0、rear=3 时有 4 个元素，已经满；front=0、rear=4 则为空。它与本章正式

SqQueue 接口是不同表示，应连同初始化、动作次序和判空判满一起使用。

不变量·尾指针不能悬空

带头结点链队列始终让 rear 指向最后一个数据结点，空时指向头结点。删去唯一数据结点后，rear 必须回到 front；否则下次入队会经已释放地址写内存。

3.4 链栈与链队列

链栈通常把链头作为栈顶，头插与头删都是 $O(1)$。动态分配可随数据量申请结点，因而通常不受预先给定的小容量数组限制；但入栈仍可能因可用内存不足而失败，出栈时也仍可能遇到空栈。

单链队列把链头作为队头，链尾作为队尾，并保存两个指针。若反过来从单链表尾部删除，必须找到尾结点的前驱，出队会变为 $O(n)$。带头结点的链队列空时 `front==rear`，二者都指向头结点。入队时接到 `rear->next` 并更新 `rear`；出队时删除 `front->next`。

```
dequeue(Q):
    if Q.front==Q.rear: return 失败
    p=Q.front->next
    x=p->data
    Q.front->next=p->next
    if Q.rear==p: Q.rear=Q.front
    释放 p
    return x
```

最后一个数据结点出队时，必须让尾指针退回头结点；否则它指向已释放的空间。非空且不止一个元素时，无需改变尾指针，所以对“删除是否修改头尾指针”这类问题，答案取决于被删除的是否为最后一个元素。

只带队首指针的非循环双链表，要在另一端入队仍需找尾部；循环双链表则可通过 `front->prior` 直接取得尾部。比较结构时，应检查有没有可到达另一端的短链接，而不是看到“双链”就认定所有操作都为常数时间。

3.5 多维数组的地址怎样计算

数组同一维度的下标取值范围固定，每个元素大小相同。设二维数组的行下标为 $l_1 \sim u_1$，列下标为 $l_2 \sim u_2$，列数 $c = u_2 - l_2 + 1$，行数 $r = u_1 - l_1 + 1$，每元素占 w 字节，首元素地址为 b。

按行优先时，在 (i, j) 前有 $i - l_1$ 个完整行及本行的 $j - l_2$ 个元素，故

$$\mathrm{Addr}(A[i,j]) = b + [(i - l_1)c + (j - l_2)]w.$$

按列优先时，先数完整列：

$$\mathrm{Addr}(A[i,j]) = b + [(j - l_2)r + (i - l_1)]w.$$

例如 `A[2..4][-1..2]` 按行优先存放，每元素 4 字节，`A[2][-1]` 的地址为 1000。`A[4][1]` 前有 $2 \times 4 + 2 = 10$ 个元素，字节地址为 1040。下标差先得到“元素数”，最后乘元素宽度才得到字节偏移；按字编址的题则要统一成字地址再计算。

三维数组按行优先时，最右下标变化最快：`A[i][j][k]` 的相对元素位置为 $((i-l_1)n_2+(j-l_2))n_3+(k-l_3)$。可以逐维理解，不必另背一个没有下标原点的公式。

错项排除·三对角首尾行少一个

n 至少为 2 时，首行和末行各只保存 2 个位置，中间行才各 3 个。若把第 i 行之前全部按 3 个计，会从第一行就多算；先列 2+3(i−2) 再加本行偏移。

3.6 对称、三角、三对角与稀疏矩阵

压缩存储利用元素间的确定关系，避免重复保存。**对称矩阵**满足 $a_{ij}=a_{ji}$，保存一侧三角即可；访问另一侧时交换 i,j。**三角矩阵**的三角区之外，元素全部相同，通常还要额外保存这个公共常量，它未必为 0。**三对角矩阵**只在主对角线及相邻两条对角线上可能非零，共有 $3n-2$ 个需要保存的位置。

下三角按行保存、原矩阵与数组下标均从 0 开始时，位置 (i,j)（$i\ge j$）之前有 $1+2+\cdots+i=i(i+1)/2$ 个元素，故数组下标为 $i(i+1)/2+j$。若原矩阵从 1 开始而数组从 0 开始，公式相应变成 $i(i-1)/2+j-1$。推导的顺序始终是“前面完整行或列数＋本行或列内偏移”。

3.6.1 例 3-3　三对角矩阵的起止行不能一律算 3 个

例 3.6.1 (配套例题)

配套例题：100 阶三对角矩阵 M 的元素 $m_{i,j}$ $(1\le i,j\le 100)$ 按行优先依次压缩存入下标从 0 开始的一维数组 N。$m_{30,30}$ 在 N 中的下标为（　）。

A. 86　B. 87　C. 88　D. 89

解析。第一行有 2 个元素，第 2 至 29 行各有 3 个，共 $2+28\times 3=86$ 个在第 30 行之前。第 30 行依次保存 $m_{30,29},m_{30,30},m_{30,31}$，目标是本行第 2 个，因此下标为 $86+1=87$，答案为 **B**。这里 87 是从 0 开始的下标，目标是第 88 个被保存的元素。

3.6.2 例 3-4　先用对称性换到实际保存的半边

例 3.6.2 (配套例题)

配套例题：10×10 对称矩阵 M 的上三角元素 $m_{i,j}$ $(1\le i\le j\le 10)$ 按列优先存入 C 语言一维数组 N。$m_{7,2}$ 在 N 中的下标为（　）。

A. 15　B. 16　C. 22　D. 23

解析。$m_{7,2}=m_{2,7}$，实际寻找第 7 列第 2 行。前 6 列各存 $1,2,\ldots,6$ 个，共 21 个；本列目标前还有第 1 行 1 个元素，所以从 0 开始的下标为 $21+1=22$，答案为 **C**。不能先把下三角坐标直接代入上三角公式，再试图以加减 1 修正。

上三角若改为按行优先，各行长度为 $n,n-1,\ldots,1$；第 i 行之前有 $(i-1)(2n-i+2)/2$ 个保存位置。下三角若按列优先，各列长度同样递减，但列内偏移为 $i-j$。公式外形相近，决定它的是保存方向与下标原点。

稀疏矩阵的非零位置没有固定三角规律，需要连同位置一起保存。三元组顺序表记录 `(行,列,值)`，还须保存矩阵行列数与非零元数；十字链表为同一个非零元同时设置行链接和列链接。压缩后不能再按普通二维数组地址公式随机访问任意元素，查找与修改成本随表示方法

而变。

最小反例·括号计数看不到嵌套

([)] 中每一类左右括号数都相等，但第三个字符要求圆括号配对，栈顶却是方括号。配对必须遵守最近未完成对象先结束，不能在扫描后任意重排。

3.7 表达式、递归与层次处理

3.7.1 括号与表达式中的“最近一个尚未完成对象”

左括号入栈；遇右括号时，栈顶必须是对应种类的左括号，匹配后弹出；扫描完后栈必须为空。`([)]` 中括号数量相等，但遇到第一个右圆括号时栈顶为左方括号，已经失败。这个过程正好需要最后遇到且尚未配对的对象，所以选择栈。

后缀表达式中，操作数先入栈，遇二元运算符时先弹出的数是**右操作数**，后弹出的是左操作数，计算后将结果入栈。减法、除法不能颠倒。合法的二元后缀表达式在遇到每个运算符时，栈中至少有两个可用值，最终恰好留下一个结果。

3.7.2 例 3-5　弹出次序怎样影响减法

例 3.7.1 (配套例题)

配套例题：栈 S_1 保存整数，S_2 保存运算符。函数 F() 依次：（1）从 S_1 弹出操作数 a、b；（2）从 S_2 弹出运算符 op；（3）执行 b op a；（4）把结果压入 S_1。初始 S_1 自底向顶为 5、8、3、2，S_2 自底向顶为 *、−、+。调用 3 次 F() 后，S_1 栈顶值是（　）。

A. -15　　B. 15　　C. -20　　D. 20

解析。第一次弹出 2、3，用加法得到 $3+2=5$，整数栈变为 5、8、5。第二次弹出 5、8，用减法得到 $8-5=3$，栈变为 5、3。第三次得到 $5\times 3=15$，答案为 **B**。若第二次按弹出先后计算 $5-8$，就会得到错误的负数。

中缀转后缀可用运算符栈：操作数直接输出；左括号入栈；右括号触发输出直到匹配左括号；遇运算符时，先输出栈顶中优先级更高的运算符。对通常左结合的 +、−、*、/，同级也先弹出；若加入右结合运算符，同优先级的处理规则需要调整。

例如 $x+y(z-u)/v$（其中相邻表示乘法）先把括号内写成 `zu-`，乘除按左结合形成 `yzu-*v/`，最后与 x 相加，得 `xyzu-*v/+`。这里的输出顺序表达运算依赖，而不是把原式的符号按种类重新排列。

3.7.3 递归返回前保存了什么

递归调用栈保存尚未完成的调用。调用树的一个结点表示一次调用，不能把函数返回值当作调用次数。对于

```
int F(int n) {
    if (n<=3) return 1;
    return F(n-2)+F(n-4)+1;
}
```

令 $C(n)$ 表示含当前调用在内的调用次数，则 $n\le 3$ 时 $C(n)=1$，否则 $C(n)=1+C(n-2)+C(n-4)$。因此 $C(4)=C(2)+C(0)+1=3$，$C(6)=1+3+1=5$，$C(8)=1+5+3=9$。式中返回值的“+ 1”不

是另一次函数调用；次数递推中的 “+ 1” 是当前调用本身。此过程对应〖3.3-006〗的递归调用模型。

C/C++ 对加号两侧函数调用的求值先后并不一概保证从左到右。若题目问 “第几个被调用”，应检查答案是否依赖这个先后，不能把一种习惯顺序当作语言保证。调用总数与最终结果在本例不受这个先后影响。另如〖3.3-007〗把递归式改为 `func(x-2)+func(x-4)`、基例返回 2，并计算 `func(func(5))`：必须先完成内层 func(5)，它和 func(3)、func(1) 共 3 次调用，结果为 4，所以第 4 次是外层 func(4)；内层两个基例谁先执行不影响这个结论。

层次处理需要先进先出的队列：先发现的一层结点必须先扩展，后发现的下一层排在后面。第 5 章层序遍历与第 6 章 BFS 都用这条约束，但图还需 `visited` 防止同一顶点反复入队。

辨析·调用次数与同时深度

Fibonacci 调用树的结点总数反映总调用次数；栈空间取同时尚未返回的调用链长度。

3.7.4 阶乘、Fibonacci 与重复子问题

下面两段保留课件的递归定义。输入 n 为非负整数，计算结果须在 int 可表示范围内。

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    else return n * factorial(n - 1);
}
int Fib(int n) {
    if (n == 0) return 0;
    else if (n == 1) return 1;
    else return Fib(n - 1) + Fib(n - 2);
}
```

factorial(10) 沿 10、9、…、1 依次深入，最多同时保留 10 层调用，随后按相反次序返回并完成乘法。Fib(4) 的调用树有 9 个结点，返回值为 3，最长未返回调用链却只有 4 层；Fib(2) 重复出现两次。朴素 Fib 递归的时间满足 $T(n) = T(n-1) + T(n-2) + O(1)$，呈指数增长，递归工作栈为 $O(n)$。每层传递两个递归分支，不意味着空间也按整棵调用树相加；一个分支返回后，其空间可供另一个使用。保存已算子问题或改用迭代可减少重复计算，改变算法后须重新分析代价。

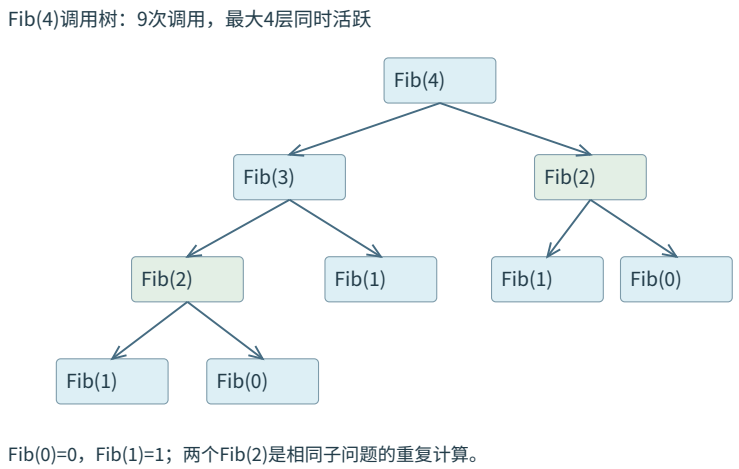


Figure 3.2: Fib(4) 的完整调用树：重复子问题与最大同时活跃调用链

3.8 栈与队列的教材接口

定义 3.8.1 (先约定指针指向什么，再写边界)

顺序栈中的 top 可以指栈顶元素，也可以指下一空位；循环队列中的 front/rear 也有不同语义。表示约定决定空满条件、增减时机和长度公式。同一个变量名并不保证同一种表示，改题时先画空状态和单元素状态。

不变量·top 含义决定增减位置

top 指栈顶时，空状态为 -1，入栈先加再写；top 指下一空位时，空状态为 0，入栈先写再加。用空栈和单元素两幅状态检查，比只背 push 语句更能应对改约定题。

下面采用教材的 `SqStack`，`top` 指当前栈顶；下标从 0 开始，空栈 `top=-1`。`ElemType` 按具体应用定义为元素类型，下面可取整型。

```
#define MaxSize 100
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int top;
} SqStack;
void InitStack(SqStack &S) { S.top = -1; }
bool StackEmpty(SqStack S) { return S.top == -1; }
bool Push(SqStack &S, ElemType x) {
    if (S.top == MaxSize - 1) return false;
    S.data[++S.top] = x;
    return true;
}
bool Pop(SqStack &S, ElemType &x) {
    if (S.top == -1) return false;
    x = S.data[S.top--];
    return true;
}
bool GetTop(SqStack S, ElemType &x) {
    if (S.top == -1) return false;
    x = S.data[S.top];
    return true;
}
```

读栈顶不改变 top，出栈才将 top 减 1。若栈顶指向下一空位，压入顺序应为 `data[top++]=x`，弹出则为 `x=data[--top]`；这是一套独立表示，不能只改初始化而保留其余操作。

循环队列使用教材的 `SqQueue`，front 指队头元素、rear 指下一入队位置，牺牲一个单元。

```c
typedef struct {
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
void InitQueue(SqQueue &Q) { Q.front = Q.rear = 0; }
bool QueueEmpty(SqQueue Q) { return Q.front == Q.rear; }
bool EnQueue(SqQueue &Q, ElemType x) {
    if ((Q.rear + 1) % MaxSize == Q.front) return false;
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;
    return true;
}
bool DeQueue(SqQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    return true;
}
bool GetHead(SqQueue Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    x = Q.data[Q.front];
    return true;
}
```

不变量·先换到已保存半边

对称矩阵访问未存的半边时，先交换 i、j，再按保存方向计数。坐标转换解决的是表示对应；数组从 0 还是 1 起始解决的是编号，两步不能互相替代。

入队先写 rear 位置再推进 rear，出队先取 front 位置再推进 front。空满检查均在修改之前进行，失败操作保持原状态，输出参数也不赋新值。读队头只读取，不出队。

环形长度公式来自有向距离

从 front 沿数组下标递增方向走到 rear，步数就是有效元素数。未绕回时为 rear−front，绕回时为 M−front+rear，合并为 $(rear - front + M) \bmod M$。牺牲一个单元将步数限制在 0 至 M−1，从而只让 0 表示空；若允许 M 个元素，模运算会把长度 M 也变成 0，必须增加 size 或标志保存这一个区别。

□

环形队列策略	空条件	满条件	可用容量
牺牲一个单元	front=rear	(rear+1) mod M=front	M−1
增加 size 计数	size=0	size=M	M
增加最近操作 tag	front=rear 且 tag 为出队	front=rear 且 tag 为入队	M

tag 方案初始设为 “出队” 态，初始化即表示空；成功入队/出队后更新 tag。计数方案每次成功修改长度，失败操作不能改 size。链队列没有预设数组容量，但申请新结点仍可能失败，不能将 “不判数组满” 理解为内存无限。

3.9 矩阵压缩：由坐标到下标，再从下标恢复坐标

完整行数加行内偏移

每次先写清：矩阵坐标从 0 还是 1 开始、压缩数组从 0 还是 1 开始、上/下三角、行/列优先。若是对称矩阵，先把目标换到实际保存的半边。最后数前面完整行或列，再加本行/列内偏移；不要用最后加减 1 修补错误的保存方向。

对于坐标从 0 开始、按行保存下三角的矩阵，令 $T_i = i(i+1)/2$，则 $k = T_i + j$。反向给定有效数组下标 k 时，先找到唯一满足 $T_i \le k < T_{i+1}$ 的 i，再令 $j = k - T_i$。数学上

$$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, \qquad j = k - \frac{i(i+1)}{2}.$$

例如 k=8，因 $T_3 = 6 \le 8 < T_4 = 10$，得 $i = 3, j = 2$。实现时若用浮点平方根，应以整数三角数边界复核，防止舍入把边界分到相邻行。

坐标从 1 开始、压缩数组从 0 开始时，上三角行优先下标为

$$k = \frac{(i-1)(2n-i+2)}{2} + (j-i), \quad 1 \le i \le j \le n.$$

下三角列优先则为

$$k = \frac{(j-1)(2n-j+2)}{2} + (i-j), \quad 1 \le j \le i \le n.$$

三角矩阵的公共常量区可另存一个值，所需空间为 $n(n+1)/2+1$；对称矩阵只保存一侧即可，不需要这个公共常量单元。三对角矩阵 $n \ge 2$ 时按行存 3n−2 个有效位置，坐标从 1、数组从 0 的统一公式为 $k = 2i + j - 3$，前提是 $|i-j| \le 1$ 且坐标不越界。n=1 时只有一个元素，应按单元素矩阵处理，不再套首尾行各 2 个的解释。

稀疏矩阵中零元位置不具有固定三角规律。三元组的 “下标” 表示第几个非零记录，不能直接由坐标用普通二维地址公式得到；修改原来的零元为非零元时，可能需要插入记录并调整次序或链接。

错项排除·FIFO 只是具体策略

队列 ADT 规定先进先出，操作系统的 “就绪队列” 名称却不保证所有调度算法都按 FIFO 选择。先来先服务可借 FIFO 说明，优先级等策略需要额外排序或选择规则。

3.10 典型应用的失败条件

中缀表达式将运算符写在左右操作数之间；前缀写在两者之前，后缀写在两者之后。前缀求值从右向左扫描，遇运算符先弹出的值是左操作数、后弹出的值是右操作数；后缀从左向右扫描则先弹出右操作数。方向与角色要一起记。

例如 $x + y(z-u)/v$ 按通常左结合规则，前缀为 `+ x / * y - z u v`，后缀为 `x y z u - * v / +`。两种形式都用位置表达运算依赖而省去括号；不是简单地把所有运算符搬到字符串开头或末尾。

按尚未完成对象的次序选结构

最近进入且尚未结束的对象要先处理，用栈：括号配对、函数调用、表达式中等待右操作数的运算。最早进入且尚未服务的对象先处理，用队列：层序遍历、BFS、先来先服务。操作系统的就绪队列只是管理等待对象，实际调度策略未必总是 FIFO。

后缀表达式求值必须同时检查：遇二元运算符时至少有两个操作数、除数合法、扫描末尾恰好留下一个结果。仅仅 “最终留下一个数” 不足以排除中途栈下溢。例如 `1 + 2` 按后缀语法扫描，在加号处就没有两个数；`1 2 3 +`则扫描后还剩两个值。

例 3.10.1 (自拟：括号数量正确仍可失败)

输入 `( [ ) ]`中左右圆括号和左右方括号数量分别相等。用栈检查时，在哪一个字符首次失败?

前两个字符依次入栈，栈顶为 `[`；读到第三个字符`)`时，它要求栈顶为 `(`，因此立即失败。只统计每类括号数量看不到嵌套关系，错误发生在第三个字符，不能等扫描完再随意重新配对。

辨析·栈顶选在链头

单链表头插头删只改常数条链接。若从单链表尾部弹出，还要先寻找前驱，代价会增加。

3.11 链栈、链队列与共享栈的实现边界

链栈通常不带头结点，Ltop 指向当前栈顶，空栈为 NULL。以下采用教材的链栈结点定义，申请与释放使用 malloc/free。

```
typedef struct Linknode {
    ElemType data;
    struct Linknode *next;
} *LiStack;
bool Push(LiStack &S, ElemType x) {
    LiStack p = (LiStack)malloc(sizeof(struct Linknode));
    if (p == NULL) return false;
    p->data = x;
    p->next = S;
    S = p;
    return true;
}
bool Pop(LiStack &S, ElemType &x) {
    if (S == NULL) return false;
    LiStack p = S;
    x = p->data;
    S = p->next;
    free(p);
    return true;
}
```

链栈从链头入栈、出栈，均为 $O(1)$；若改在普通单链表尾部出栈，定位尾结点前驱会变为线性时间。初始化令 S 为空即可；销毁时重复弹出或逐结点释放。

3.11.1 带头结点链队列

头结点使空队列和一般出队的前驱统一。front 始终指头结点，rear 指最后一个结点；空时二者指向同一个头结点。

```c
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LinkNode;
typedef struct {
    LinkNode *front, *rear;
} LinkQueue;
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = (LinkNode *)malloc(sizeof(LinkNode));
    Q.front->next = NULL;
}
bool IsEmpty(LinkQueue Q) { return Q.front == Q.rear; }
void EnQueue(LinkQueue &Q, ElemType x) {
    LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));
    s->data = x;
    s->next = NULL;
    Q.rear->next = s;
    Q.rear = s;
}
bool DeQueue(LinkQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    LinkNode *p = Q.front->next;
    x = p->data;
    Q.front->next = p->next;
    if (Q.rear == p) Q.rear = Q.front;
    free(p);
    return true;
}
```

辨析·最后一删要回尾

删除唯一数据结点后，rear 必须退回 front；二者共同指向仍然有效的头结点。

初始化和入队保留王道课件的 void 接口，以内存申请成功为前提。实际程序须检查 malloc 结果，并在申请失败时保持原队列。最后一个数据结点出队后，必须把 rear 退回 front；随后再次入队才有有效尾指针。销毁链队列还需释放头结点。

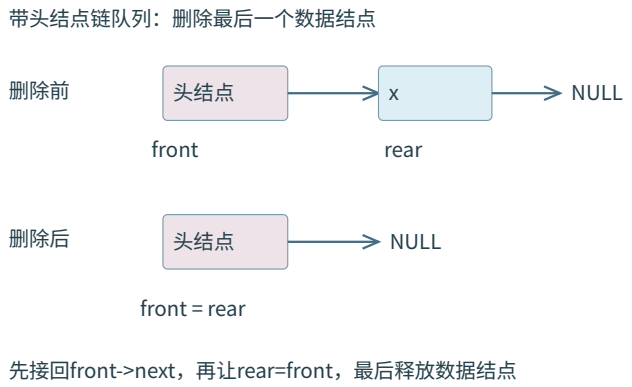


Figure 3.3: 最后一个数据结点出队后，front 与 rear 共同指向有效头结点

若采用不带头结点的版本，首次入队须同时建立 front 和 rear，最后一次出队须同时置空二者。

以下是另一套独立实现，仍使用 LinkQueue 类型，但 front 指首个数据结点。不能与前面的带头结点接口混合调用。

```
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = NULL;
}
bool IsEmpty(LinkQueue Q) { return Q.front == NULL; }
void EnQueue(LinkQueue &Q, ElemType x) {
    LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));
    s->data = x;
    s->next = NULL;
    if (Q.front == NULL)
        Q.front = Q.rear = s;
    else {
        Q.rear->next = s;
        Q.rear = s;
    }
}
bool DeQueue(LinkQueue &Q, ElemType &x) {
    if (Q.front == NULL) return false;
    LinkNode *p = Q.front;
    x = p->data;
    Q.front = p->next;
    if (Q.rear == p) Q.rear = NULL;
    free(p);
    return true;
}
```

辨析·共享不改变次序

共享栈只共享空闲容量，两栈仍各自后进先出。top1-top0=1 表示中间已无空格。

3.11.2 共享栈的容量如何协同

```
typedef struct {
    ElemType data[MaxSize];
    int top0, top1;
} ShStack;
void InitStack(ShStack &S) {
    S.top0 = -1;
    S.top1 = MaxSize;
}
```

教材 `ShStack` 的两端栈顶初值为 `top0=-1,top1=MaxSize`，共享空闲区在两栈顶之间，满条件 `top1-top0==1`。若入 0 号栈，先检查满，再令 `data[++top0]=x`；入 1 号栈则令 `data[--top1]=x`。出 0 号栈取 `data[top0--]`，出 1 号栈取 `data[top1++]`，各自先判空。

共享只提高同一块空间的利用率；两个栈仍各自满足后进先出。若一栈已占满全部单元，另一栈即使为空也不能继续入栈。

3.12 括号匹配的完整状态检查

用字符作为 ElemType，复用顺序栈接口。新左括号意味着最近一个尚未完成对象；右括号必须关闭栈顶的同种左括号。

```
bool bracketCheck(char str[], int length) {
    SqStack S;
    InitStack(S);
    for (int i = 0; i < length; ++i) {
        char c = str[i];
        if (c == '(' || c == '[' || c == '{') {
            if (!Push(S, c)) return false;
        } else if (c == ')' || c == ']' || c == '}') {
            ElemType x;
            if (!Pop(S, x)) return false;
            if ((c == ')' && x != '(') ||
                (c == ']' && x != '[') ||
                (c == '}' && x != '{')) return false;
        }
    }
    return StackEmpty(S);
}
```

辨析·相同优先级看结合性

乘除和加减通常左结合，中缀转后缀遇同级运算符时先弹栈顶。新运算符不能抢先。

遇右括号但栈为空，属于右括号过早；栈顶类型不符，属于交叉嵌套；扫描结束栈非空，属于左括号尚未关闭。每字符只处理常数次，时间 $O(n)$，最坏保留 n 个左括号，辅助空间 $O(n)$；定长栈实现还要求实际所需深度不超过容量。

3.13 按课件表达式保留完整转换过程

中缀表达式 `A+B-C*D/E+F` 的运算次序为：先 A+B，再 C*D，再除以 E，再从 A+B 减去该商，最后加 F。按左结合规则得到后缀：

```
A B + C D * E / - F +
```

当前符号	运算符栈（底到顶）	已输出后缀
A	空	A
+、B	+	A B
−	−	A B +
C、*、D	−、*	A B + C D
/	−、/	A B + C D *
E	−、/	A B + C D * E
+	+	A B + C D * E / −

当前符号	运算符栈（底到顶）	已输出后缀
F、扫描结束	空	A B + C D * E / − F +

遇到/时先弹同级 *，对应乘除左结合；遇到最后一个 +时，先弹除号再弹减号。对于 `A+B*(C-D)-E/F`，括号内的减法先完成，得到 `A B C D - * + E F / -`。左括号只作为界限保存在栈中，不进入后缀输出。

课件的数值表达式

$$((15/(7-(1+1)))\times 3)-(2+(1+1))$$

对应后缀为 `15 7 1 1 + - / 3 * 2 1 1 + + -`。依次计算 $1+1=2$、$7-2=5$、$15/5=3$、$3\times 3=9$，右部分为 $2+(1+1)=4$，结果为 5。数字 15 是一个操作数，不能按字符 1 和 5 分开入栈。模拟可知操作数栈峰值为 4 个值，这和表达式的字符总数不同。

前缀形式用 “运算符、左操作数、右操作数” 表达同一依赖。从右向左求前缀值时，先弹出的数为左操作数；从左向右求后缀值时，先弹出的数为右操作数。把两种扫描方向混用，会在减除法中得到错误结果。

辨析·三角区与常量区

对称矩阵只存半边；三角矩阵还需存另一半的公共常量。两者存储单元数相差 1。

3.13.1 直接用两个栈计算中缀表达式

也可以将 “中缀转后缀” 与 “后缀求值” 融合：操作数立即压入操作数栈；运算符和括号按转换算法管理。每当转换规则本应输出一个运算符时，直接从操作数栈先弹右操作数、再弹左操作数，计算后将结果压回。左括号仍只作边界，不参与运算；表达式结束时清理运算符栈。

以 `A+B-C*D/E+F` 为例，在读到减号时，先弹出加号并把 A+B 压回；读到除号时，先完成 *CD*；读到最后一个加号时，依次完成 *(C*D)/E 和 (A+B)−(C*D)/E；最后再加 F。这与后缀 `A B + C D * E / - F +`表达同一依赖。两个栈分别保存 “尚未归约的操作数” 和 “尚未确定何时执行的运算符”，不可把运算符与值混放在同一种栈元素模型中。

除括号配对外，还应检查操作数数量、运算符合法性、除零及最终结果数；若题目包含一元负号或多位数，应先按词法单元识别，再应用相应操作数个数的规则。

3.14 数组与特殊矩阵的定位总表

下表统一规定：原矩阵坐标从 1 开始，压缩数组 B 下标从 0 开始；n 为阶数。先依据对称性换到实际保存的三角区，再计算 k。

保存区域与次序	合法坐标	压缩下标 k
下三角行优先	$i \ge j$	$i(i-1)/2 + j - 1$
上三角列优先	$i \le j$	$j(j-1)/2 + i - 1$
上三角行优先	$i \le j$	$(i-1)(2n-i+2)/2 + j - i$
下三角列优先	$i \ge j$	$(j-1)(2n-j+2)/2 + i - j$
三对角行优先	$\lvert i-j \rvert \le 1$	$2i + j - 3$

对称矩阵保存 $n(n+1)/2$ 个元素即可。三角矩阵另一个区域是同一个公共常量，需再保存一个常量单元，共 $n(n+1)/2+1$ 个；公共常量不必为 0。三对角矩阵 n≥2 时保存 $3n-2$ 个元素，首尾行各 2 个，中间每行 3 个。n=1 时只有一个有效位置，应直接按单元素处理。

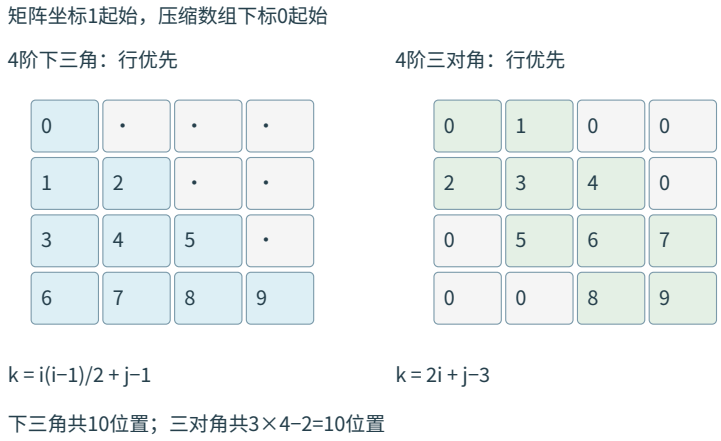


Figure 3.4: 4 阶下三角与三对角矩阵的压缩位置，矩阵 1 起始、数组 0 起始

反向给定三对角压缩下标 k 时，第 i 行之前有 $3(i-1)-1$ 个保存位置（首行单独核对），第 i 行结束前共 $3i-1$ 个。因此可以取

$$i=\left\lceil \frac{k+2}{3} \right\rceil, \qquad j=k-2i+3.$$

行号也可写为 $i=\lfloor (k+1)/3 \rfloor+1$，与上述向上取整形式等价；应先确定 i 再求 j。

例如 k=87，$i=\lceil 89/3 \rceil=30$，$j=87-60+3=30$，与例 3-3 一致。前提是 $0\le k<3n-2$ 且 n≥2；得到坐标后还可复核 $|i-j|\le 1$。

课件的 5 行 6 列稀疏矩阵有六个非零项，按行次序的三元组为：

行 i	列 j	值 v
1	3	4
1	6	5
2	2	3
2	4	9
3	5	7
4	2	2

第 5 行全零，仍属于矩阵；只看非零记录无法恢复总行数，所以三元组表还要记录矩阵总行数、总列数和非零元数。十字链表中同一个非零结点同时进入行链和列链，修改时须维护两套联系。稀疏仅说明非零元少，不能据此使用三角矩阵的固定位置公式。

自查提示·队列的满空条件

写清采用牺牲一个单元、计数器还是标志位，才能计算容量并判断队满。

3.15 主动回忆自测

第 2 题引自王道书，其余为原创补充，检查表示约定。

1. 数组有 8 个单元、牺牲一个单元的循环队列，`front=6,rear=2`，长度是多少？还能再入队几个元素？
2. 〖3.1-025〗顺序共享栈 `Share[0..n-1]`，第一个栈顶 top1 初值 −1，第二个栈顶 top2 初值 n，栈满条件是（　）。A. `top2-top1==1`　B. `top1-top2==1`　C. `top1==top2`　D. 都不对
3. 下标从 1 开始的三对角矩阵，若压缩数组也从 1 开始，非边界行元素 $a_{i,j}$ 的位置如何计算？

答案。1. 长度 $(2-6+8) \bmod 8 = 4$，最大容量 7，还能入队 3 个。2. **A**，两栈顶相邻时已无可用单元，不能等栈顶重合才判满。3. 前面有 $2+3(i-2) = 3i-4$ 个元素，本行从 $j = i-1$ 开始，故从 1 开始的位置为 $3i-4+[j-(i-1)]+1 = 2i+j-2$；前提为 $2 \le i \le n-1$ 且 $|i-j| \le 1$，首尾行须按实际保存位置解释。

自查提示·队列的满空条件

写清采用牺牲一个单元、计数器还是标志位，才能计算容量并判断队满。

3.15.1 补充自拟回忆题

先画状态，再作判断

4. 入栈顺序为 a、b、c，输出 c、a、b 是否合法？第一次失败时栈顶是谁？
5. 带头结点链队列删除最后一个数据结点时，front 和 rear 应指向哪里？
6. 同一个 n 阶对称矩阵，为什么下三角行优先和上三角列优先有相同的三角数结构？坐标如何转换？
7. 后缀表达式 `8 3 - 2 /`的值是多少？若按整数除法计算，先弹出的数是什么角色？
8. 递归访问 n 个结点，最大同时活跃调用数一定为 n 吗？应由哪个结构参数决定？

补充题核对答案

4. 不合法。弹出 c 后栈中为 a、b，想输出 a 时栈顶是 b，而输入已用尽。
5. front 仍指有效头结点，rear 必须退回 front；头结点的 next 为空。rear 不能留在已释放的数据结点上。
6. 一边各行保存 1、2、⋯ 个元素，另一边各列也保存 1、2、⋯ 个。先用 aij=aji 交换坐标，再按对应行或列求偏移。
7. 先计算 8−3=5，再作 5/2，整数结果为 2；每次先弹出右操作数，除法的除数是先弹出的 2。
8. 不一定。由最长未返回调用链决定；树遍历通常取树高 h，最坏链形树才可达到 n。